



Why Vite ❤️ Javascript Monorepos

Ema Zyka

Principal Software Engineer / **allwyn** • LOTTERY SOLUTIONS

Co-organizer / Vue.js Athens meetup 

 emazyka  emazyka



What is a Monorepo?

A single git repository containing multiple packages

Polyrepo Pain

- Change in library A
- Update all dependent repos manually
- Run NPM install everywhere
- Fall behind on updates

Monorepo Solution

- Single PR touches multiple packages
- No follow-up steps needed
- Run entire test suite easily
- Keep everything in sync



When to Choose Monorepo?

Key indicators for your decision

Choose Monorepo When:

- High package linkage
- Sophisticated shared tooling
- Pain keeping teams upgraded
- Need unified conventions

Stick with Polyrepo When:

- Self-contained libraries
- Simple CLI tools
- Setup overhead not worth it
- Independent projects

The Monorepo Pain Points

Common challenges that slow down development

Slow Builds

- Rebuilding everything on changes
- No incremental builds
- Long CI/CD pipelines

Complex Setup

- Multiple build tools
- Inconsistent configs
- Dependency hell

Development Friction

- Slow HMR across packages
- Hard to debug shared code
- Inconsistent dev experience



Great tool for modern Javascript monorepos

Lightning Fast

- Native ESM in dev
- Instant HMR
- Optimized builds

Monorepo Ready

- Workspace support
- Shared dependencies
- Unified configs

Framework Agnostic

- React, Vue, Svelte, vanilla JS
- Same tooling across apps
- Consistent DX



Monorepo Structure

```
game-monorepo/  
├── packages/  
│   ├── game-client/    # React + Pixi.js UI  
│   ├── game-server/    # Node.js + Socket.io  
│   └── game-shared/     # Types & utilities  
├── package.json         # Root workspace config  
└── tsconfig.json        # Shared TypeScript config
```

📦 3 main packages + shared configs

🚀 Simple but production-ready

🎯 Easy to understand and maintain

Vite Configuration

Simple workspace setup with shared dependencies

```
// package.json (root)
{
  "workspaces": ["packages/*"],
  "scripts": {
    "dev": "npm run dev --workspace=packages/game-client",
    "build": "npm run build --workspace=packages/game-client"
  }
}
```

```
// packages/game-client/vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  optimizeDeps: {
    include: ['pixi.js', 'socket.io-client']
  },
  resolve: {
    alias: {
      '@game-shared': resolve(__dirname, '../game-shared')
    }
  }
})
```

Shared Dependencies & Optimization

One version across all packages with intelligent bundling

```
// package.json (root)
{
  "dependencies": {
    "pixi.js": "^8.0.0"
  }
}
```

```
// packages/game-shared/src/index.ts
import * as PIXI from 'pixi.js'

export function createGameContainer(): PIXI.Container {
  return new PIXI.Container()
}

export interface Player {
  id: string
  position: { x: number, y: number }
}
```

 Single source of truth for dependencies

 Vite pre-bundles shared packages

 Optimized chunk splitting



Workspace Dependencies

Managing internal package dependencies

```
// packages/game-engine/package.json
{
  "name": "@game/engine",
  "version": "1.0.0",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "dependencies": {
    "pixi.js": "^8.0.0"
  },
  "peerDependencies": {
    "react": "^18.0.0"
  }
}
```

```
// apps/game-client/package.json
{
  "name": "game-client",
  "dependencies": {
    "@game/engine": "workspace:*",
    "@game/shared": "workspace:*",
    "react": "^18.0.0"
  }
}
```



Shared Architecture

ECS system and abstractions shared across packages

```
// packages/game-shared/src/types.ts
export interface Player {
  id: string
  position: { x: number, y: number }
  score: number
}
```

```
export interface GameState {
  players: Player[]
  currentLevel: number
}
```

```
// packages/game-shared/src/pixi-utils.ts
import * as PIXI from 'pixi.js'
```

```
export function createGameContainer(): PIXI.Container {
  const container = new PIXI.Container()
  container.sortableChildren = true
  return container
}
```

```
export function createSprite(texture: PIXI.Texture): PIXI.Sprite {
  const sprite = new PIXI.Sprite(texture)
```

⚡ Turbo vs NX: The Build Tool Battle

Both excel at cached task execution

Cached Tasks Concept

- Pure functions: same input/output
- Skip repeated work (linting, testing)
- Fingerprint inputs to cache outputs
- Massive speed improvements

Cloud Caching

- Share cache across team
- CI/CD benefits from cached results
- Distributed build acceleration
- Both tools offer this feature

⚡ Incredible speedup for deterministic tasks

☁ Team-wide cache sharing

🔄 Both tools work similarly well

Development Workflow

Simple commands for the entire monorepo

```
// package.json (root)
{
  "scripts": {
    "dev": "npm run dev --workspace=packages/game-client",
    "dev:server": "npm run dev --workspace=packages/game-server",
    "build": "npm run build --workspaces --if-present",
    "test": "npm run test --workspaces --if-present"
  }
}
```

```
# How to run the project
npm i
npm run dev
```

-  Simple workspace commands
-  Vite dev server + Node.js server
-  Unified build and test processes



Team Collaboration & Updates

Consistent experience and easy dependency management

Team Benefits

- Install at root
- Root scripts start everything
- Share components via packages
- Same tooling for everyone

Easy Updates

- Updates all apps automatically
- Rebuilds affected packages
- Consistent versions everywhere

Thank You!!



@emazyka

Slides on Github